

Supervision Trees — A Readable Companion to the OTP Docs

Sample chapter · checked against OTP 29.1.1 · Sep 25, 2026

1. Why supervision exists

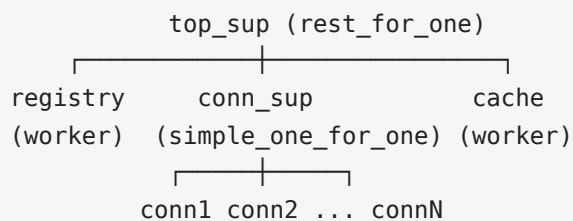
A supervisor turns an unknown crash into a known state: it restarts a failed process from a clean initial state instead of trying to repair it. That is the whole idea; everything else is configuration.

Most bugs that survive testing are transient: a race, a timeout, an odd input, a dependency briefly down. Defensive code tries to anticipate each one and usually handles them badly. Erlang inverts this. A worker handles only the cases it understands and crashes on the rest. The supervisor, which does nothing but watch, restarts it.

This works only because Erlang processes share no memory. A crash cannot corrupt a neighbour's state, so discarding one process and starting a fresh one is safe. In a shared-memory language, the same restart would leave you guessing what the dead thread left half-written.

The error kernel

Arrange processes so that the parts which must not fail are small and simple, and the risky parts sit at the leaves. Supervisors form the tree; workers do the real work. Put state you cannot afford to lose near the root, or outside process memory entirely (ETS owned by a stable process, a database, disk).



Read it left to right as a dependency order: the registry starts first because everything after it relies on it. That ordering is what `rest_for_one` exploits, as section 2 shows.

What “let it crash” does not mean

It does not mean ignoring errors. Expected failures, such as a user typing a bad password or a file not existing, are ordinary return values. You crash on the unexpected: states your code has no correct answer for. A restart only helps if the fresh state has a real chance of succeeding; a process that crashes on every start just burns the restart budget and takes its supervisor down.

2. The building blocks

A supervisor is fully described by one callback, `init/1`, returning `{ok, {SupFlags, ChildSpecs}}`. Both are maps; every key except `id` and `start` has a default, so a minimal spec is two lines.

```
init([]) ->
  SupFlags = #{strategy => rest_for_one, intensity => 5, period => 30},
  Children = [
    #{id => registry, start => {registry, start_link, []}},
    #{id => conn_sup, start => {conn_sup, start_link, []}, type => supervisor},
    #{id => cache, start => {cache, start_link, []}, shutdown => 2000}
  ],
  {ok, {SupFlags, Children}}.
```

Restart strategies: who dies with whom

The strategy encodes dependencies between siblings. Children start left to right and are terminated right to left.

Strategy	When one child dies	Use when
<code>one_for_one</code> (default)	Only that child restarts	Siblings are independent
<code>rest_for_one</code>	It and every child started after it restart	Later children depend on earlier ones
<code>one_for_all</code>	All children restart	Siblings share state and cannot run without each other
<code>simple_one_for_one</code>	Only that child restarts	Many dynamic instances of one child type

In the example above, if `registry` dies, `conn_sup` and `cache` restart too, because they may hold references to the old `registry`. If `cache` dies, nothing else is touched.

Restart types: does this child come back?

restart	Restarted after	Typical child
<code>permanent</code> (default)	Any exit	Long-lived servers
<code>transient</code>	Abnormal exit only: not <code>normal</code> , <code>shutdown</code> , or <code>{shutdown, Term}</code>	A job that should finish once but retry on failure
<code>temporary</code>	Never, not even when a sibling's death takes it down	Fire-and-forget tasks, per-request processes

A temporary child's spec is deleted as soon as it exits, so `restart_child/2` cannot bring it back.

Restart intensity: when the supervisor gives up

If more than `intensity` restarts happen within `period` seconds, the supervisor kills all its children and exits with reason `shutdown`. Its parent then applies its own strategy. The defaults are 1 restart per 5 seconds.

Think of the two numbers as a burst size and a sustained rate: burst = intensity, sustained rate $\approx$ intensity / period.

So 5/30 tolerates a burst of 5 but at most one restart per 6 seconds long-term, while 1/6 has the same sustained rate but cannot survive two crashes in a row. Restarts also multiply down the tree: a child under two supervisors each allowing 10 restarts can be restarted about 100 times before the application dies.

Shutdown: how a child is stopped

shutdown	What the supervisor does	Default for
<code>brutal_kill</code>	Sends <code>kill</code> immediately	None
Integer (ms)	Sends <code>shutdown</code> , waits, then sends <code>kill</code>	Workers: 5000
<code>infinity</code>	Sends <code>shutdown</code> , waits forever	Supervisors

A child supervisor must use `infinity`; a finite value can kill it midway through stopping its own children. For a worker, `infinity` makes the whole tree's shutdown depend on that worker's `terminate/2` always returning.

Automatic shutdown (OTP 24+)

A supervisor can represent one unit of work and stop itself when that work is done. Set `auto_shutdown => any_significant` or `all_significant`, and mark the relevant `transient` or `temporary` children `significant => true`. The children just exit normally; they need no knowledge of the tree. Keep such supervisors non-permanent in their parent, or the parent will restart them straight back into shutting down.

3. How the runtime actually does it

Supervisors are ordinary Erlang code built on three VM primitives: links, exit signals, and trapping exits. Knowing these explains every surprising behaviour in section 4.

Links and exit signals

A link is a two-way bond between processes. When one dies, the VM sends an exit signal carrying its exit reason to every linked process. What the receiver does depends on the reason and on whether it traps exits:

Signal reason	Receiver not trapping exits	Receiver trapping exits
normal	Ignored	Gets {'EXIT', From, normal} message
Any other reason	Dies with the same reason	Gets {'EXIT', From, Reason} message
kill (sent explicitly)	Dies with reason killed	Dies with reason killed: cannot be trapped

A supervisor traps exits. That single flag is what turns a child’s death from something that kills the supervisor into a message it can act on.

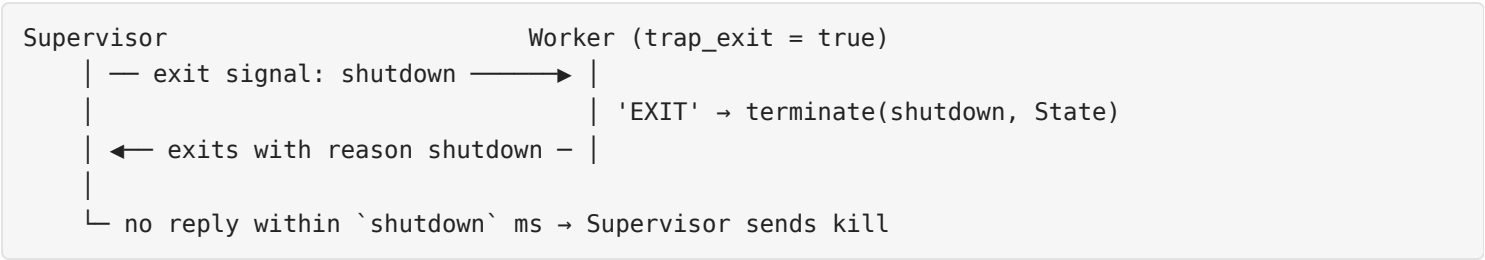
Starting: synchronous and in order

Each child’s start function must spawn, link, and return {ok, Pid}. The OTP start_link functions do this through proc_lib, which blocks the caller until the child’s init/1 returns. Consequences:

- Children start strictly in list order; child 2’s init/1 can rely on child 1 being up.
- A slow init/1 blocks the supervisor, and so the whole boot sequence.
- If any child fails to start at boot, the supervisor stops the ones already running and fails with {error, {shutdown, Reason}}.
- Returning ignore from init/1 is allowed: the spec is kept (unless temporary) with no process behind it.

Restarts and terminate_child/2 also run inside the supervisor process. While one is in progress, the supervisor answers nothing else.

Stopping: the shutdown protocol



The supervisor sends shutdown, then waits for the link’s ‘EXIT’ back, and escalates to kill on timeout. Children are stopped in reverse start order; simple_one_for_one children are stopped in parallel, in no defined order.

The detail people miss: a gen_server only runs terminate/2 on shutdown if it traps exits. Without process_flag(trap_exit, true) in init/1, the shutdown signal kills it directly, and cleanup code never runs. With brutal_kill, terminate/2 never runs either way.

Why restarts are safe

A restarted child is a new process with a new pid and a state built from scratch by init/1. Nothing carries over except what lives outside the process: registered names, ETS tables owned by others,

external systems. Anyone holding the old pid now holds a dead reference, which is why sections 2 and 4 keep pointing at dependency order and names.

4. Plausible but wrong

Each snippet below compiles, passes a happy-path test, and is the kind of code an assistant produces readily. Each fails only under a crash or a shutdown.

4.1 Cleanup that never runs

```
init(Path) ->
  {ok, Fd} = file:open(Path, [append]),
  {ok, #{fd => Fd}}.

terminate(_Reason, #{fd := Fd}) ->
  file:sync(Fd), file:close(Fd).
```

On shutdown the supervisor sends an exit signal, not a message. The process does not trap exits, so it dies instantly and `terminate/2` is skipped; buffered data is lost. Fix: call `process_flag(trap_exit, true)` first in `init/1`, and give the child an integer `shutdown`, not `brutal_kill`.

4.2 A supervisor child that forgot its type

```
#{id => conn_sup, start => {conn_sup, start_link, []}}
```

`type` defaults to `worker`, and a worker's `shutdown` defaults to 5000 ms. A busy subtree that needs longer is killed mid-shutdown and can leave orphaned grandchildren. Fix: `type => supervisor`, which also makes `shutdown` default to `infinity`.

4.3 Connecting in `init/1`

```
init(Opts) ->
  {ok, Conn} = db:connect(Opts),    % may take seconds, may fail
  {ok, #{conn => Conn}}.
```

`init/1` blocks the supervisor and every sibling after it. If the database is down, the child crashes on each start and exhausts the default 1 restart per 5 s almost immediately, taking the supervisor, then its parent, then the application down. A dependency outage becomes a node outage. Fix: return quickly with `{ok, State, {continue, connect}}`, connect in `handle_continue/2`, and retry with backoff while reporting “not ready” to callers.

4.4 Holding a sibling's old pid

```
%% cache.erl, under a one_for_one supervisor
init([]) ->
    Reg = whereis(registry),
    {ok, #{registry => Reg}}.
```

When `registry` restarts it gets a new pid. `cache` keeps sending to the dead one: `gen_server:call` exits with `noproc`, and plain `!` fails silently. Fix: address it by registered name on every call, or monitor it, or encode the dependency with `rest_for_one` so `cache` restarts after `registry`.

4.5 Permanent per-connection workers

```
init([]) ->
    {ok, #{strategy => simple_one_for_one,
          [{id => conn, start => {conn, start_link, []}}]}}.
```

The child defaults to `permanent` and the supervisor to 1 restart per 5 s. Two clients sending bad input within 5 seconds kill the supervisor and every healthy connection with it. A restarted connection process has no socket anyway. Fix: `restart => temporary` for per-request or per-connection processes; let the client reconnect.

4.6 Stopping your own supervisor

```
handle_info(job_done, State) ->
    supervisor:terminate_child(parent_sup, my_sup),
    {noreply, State}.
```

A process inside `my_sup` asks the parent to stop `my_sup`. The parent waits for the subtree to exit; the subtree waits for this process; this process waits for the call to return. If this process traps exits, it deadlocks until its shutdown timeout fires; if not, it is killed mid-call. Fix: mark the process `significant` in a supervisor with `auto_shutdown`, and just return `{stop, normal, State}`.

5. Review checklist and sources

Use this when reviewing a supervisor, whether you wrote it or an assistant did.

- ☐ Child order matches dependency order, and the strategy reflects it (`rest_for_one` where later children need earlier ones)
- ☐ Every child supervisor has `type => supervisor`
- ☐ Workers with cleanup trap exits and have an integer `shutdown`
- ☐ `init/1` returns fast; slow setup is in `handle_continue/2`
- ☐ Per-request and per-connection children are `temporary`
- ☐ `intensity/period` are set deliberately: burst size and sustained rate, not identical at every level
- ☐ No process holds a sibling's pid across a possible restart
- ☐ Work-unit supervisors use `auto_shutdown` and are not `permanent` in their parent

Sources

- Supervisor Behaviour, OTP Design Principles: https://www.erlang.org/doc/system/sup_princ.html
- supervisor module reference, stdlib: <https://www.erlang.org/doc/apps/stdlib/supervisor.html>